

1 Scenario: Parsing Table Conflict in LL(1)

In an LL(1) parser, the parsing table is constructed using the First and Follow sets of the grammar, while the Follow set contains terminals that can appear immediately after a non-terminal. During parsing, the production is placed in the table based on sets on those sets.

A conflict occurs when more than one production is entered into the same-table cell for a non-terminal and an input symbol. This usually happens because the grammar contains left recursion, common prefixes (left factoring problem).

Decision: The grammar should be modified by eliminating left recursion, performing left factoring and ensuring that the First and Follow sets do not overlap. This converts the grammar into a valid LL(1) grammar with a conflict-free parsing table.

2) Scenario: Incorrect Parse Tree Generation

The syntax analyzer constructs a parse tree by applying grammar productions accordingly to the input tokens for the expression $id + id - id$. The parse tree determines the order in which operations are evaluated.

If the grammar does not specify operator associativity the expression as $id + (id - id)$ or correct $(id + id) - id$. This results in 'incorrect' evaluation because the operators are not associated properly.

Decision: The grammar should be left-associative.
For example

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow id$$

Scenario: Invalid Expression Handling

The lexical analyzer correctly identifies the tokens. In the expression $a + * b$ as identifiers and operators.

In Predictive Parsing (LL). The parser detects the error when no valid productions exist in the parsing table for the current input symbol. In shift-reduce parsing (LR) the parser detects error when no valid shift or reduce actions is available.

Without proper recovery, the parser stops after reporting the error.

Decision: The parser should implement detection and recovery techniques such as panic-mode recovery, phrase-level recovery, Follow sets. These methods allow the parser to continue analyzing the remaining input while providing meaningful error messages.

4. scenario. Ambiguity in Conditional statements

The Grammar

$$S \rightarrow \text{if } E \cdot \text{then } s$$

$$S \rightarrow \text{if } E \cdot \text{then } \cdot s \text{ else } S$$

$$S \rightarrow a.$$

is ambiguous because the parser cannot determine which if statement else belongs to. This is known as the dangling else problem. When constructing the parsing table, conflicts arise because the both productions with the same prefix.

Decision: The Grammar should be revised to distinguish matched and unmatched statements for example.

$$S \rightarrow M|U$$

$$M \rightarrow \text{if } E \cdot \text{then } M \cdot \text{else } M|a$$

$$U \rightarrow \text{if } E \cdot \text{then } S|\text{if } \cdot \text{then } M \text{ else } \cdot U$$

5. Scenario: Operator Precedence Conflict

The grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

is ambiguous because it does not define operator precedence or associativity.

For the input $id + id * id$ two different trees

- $(id + id) * id$
- $(id) + (id * id)$

Operator precedence and associativity.

Decision: The grammar should be written

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow id$$